

PROJETO INTEGRADOR · ADS 4 PERIODO · N2

PITCH TECNICO

Defesa das Escolhas Tecnologicas e Arquiteturais

TeeStore — E-Commerce de Camisetas

Java 21 + Spring Boot	Apache Kafka	Arquitetura Modular	Virtual Threads
WebSocket STOMP	JWT + Redis	React + Vite	Design Patterns

Autores	Victor Hugo · Josue Felix · Guilherme Bastos
Curso	Analise e Desenvolvimento de Sistemas — 4 Periodo
Instituicao	PUC Goias · Semestre 2026/1

Introducao — O que este documento defende

Este pitch tecnico documenta a **justificativa racional por tras de cada decisao arquitetural e tecnologica** do projeto TeeStore. O documento norteador exige um ambiente de desenvolvimento complexo com mensageria obrigatoria — CRUD simples nao e aceito. Cada escolha tecnologica foi feita com proposito, nao por inercia.

A estrutura segue o modelo ADR (Architecture Decision Record): contexto (qual problema existia), decisao tomada, alternativas consideradas e consequencias (vantagens e desvantagens).

Criterio atendido: Criterio 4 da N2 — Apresentacao: Pitch tecnico defendendo as escolhas tecnologicas e arquiteturais.

DECISAO
01**Java 21 + Spring Boot 3.x**

Contexto: O projeto precisa de um backend com suporte a Kafka, WebSocket, JPA e segurança, dentro do contexto acadêmico de ADS que já usa Java como linguagem principal.

Decisão tomada: **Java 21 com Spring Boot 3.x, aproveitando Virtual Threads (Project Loom) e Records.**

Alternativas descartadas:

Alternativa	Por que foi descartada
Quarkus	Curva de aprendizado alta para a equipe; menos material didático no curso.
Node.js + NestJS	Suporte fraco a Kafka e JPA; equipe sem background em JS backend.
Python + FastAPI	Ecossistema Kafka menos maduro; sem JPA equivalente; tipagem opcional gera bugs.
Spring Boot 2.x	Não suporta Virtual Threads; versão em fim de suporte estendido.

Vantagens	Desvantagens / Mitigações
+ Ecossistema Kafka maduro (spring-kafka) — integração nativa	- Configuração mais verbosa que frameworks modernos — mitigado pelo autoconfigure do Spring Boot
+ Virtual Threads nativos: Logistics Service processa pedidos sem esgotar threads de SO	
+ Records como DTOs imutáveis — zero boilerplate, thread-safe por natureza	
+ Equipe já domina Java — sem curva de aprendizado adicional	
+ Spring Security + JWT bem integrado — autenticação robusta com poucas linhas	
Veredicto: Java 21 + Spring Boot 3.x é a escolha correta. Maturidade, ecossistema e domínio da equipe superam qualquer alternativa no contexto deste projeto.	

```
# Virtual Threads – application.yml (1 linha)
spring.threads.virtual.enabled: true

# Record como DTO imutável (DTOs do projeto)
public record OrderCreatedEvent(
    UUID orderId, UUID userId,
    List items, BigDecimal total) {}
```

DECISAO
02**Apache Kafka — Mensageria Assincrona**

Contexto: O documento norteador exige mensageria como diferencial técnico obrigatório. O processamento de pedidos não deve bloquear a resposta ao cliente nem acoplar o Backend ao Logistics Service.

Decisão tomada: **Apache Kafka com cluster de 3 brokers, 3 tópicos distintos e consumer groups isolados.**

Alternativas descartadas:

Alternativa	Por que foi descartada
REST síncrono entre serviços	Alto acoplamento: Backend precisaria conhecer o endpoint do Logistics. Se o Logistics cair, o pedido falha.
RabbitMQ	Mensagens destruídas após consumo — sem replay. Kafka retém por tempo configurável.
Redis Pub/Sub	Sem persistência de mensagens; sem consumer groups; sem replay — inadequado para pedidos críticos.

Vantagens	Desvantagens / Mitigações
+ Desacoplamento total: Backend não conhece Logistics nem Notification	- Overhead de configuração (Zookeeper + brokers) — aceito dado o contexto educacional
+ At-least-once delivery: mensagem nunca se perde mesmo se consumidor cair	- Ordem garantida apenas dentro da mesma partição — mitigado com 1 partição em dev
+ Replay: eventos podem ser reprocessados — útil em bugs de produção	
+ Consumer groups: Logistics e Notification consomem independentemente o mesmo tópico	
+ Cluster 3 brokers: alta disponibilidade, tolerância a falha de 1 broker	
Veredito: Kafka é a única escolha que atende ao requisito de mensageria desacoplada do roteador. REST síncrono violaria o critério de 0,4 pts da N2.	

Critério	REST Síncrono	Apache Kafka (escolhido)
Acoplamento	Alto — produtor conhece endpoint do consumidor	Baixo — produtor só conhece o tópico
Disponibilidade	Consumidor offline = erro imediato no produtor	Mensagem retida no broker até consumidor subir
Desempenho	Bloqueante — aguarda processamento logístico	Não-bloqueante — Backend retorna 201 imediatamente
Rastreabilidade	Sem histórico de chamadas	Eventos persistidos e reprocessáveis (replay)
UX (tempo real)	Requer polling no frontend	WebSocket push — 0 polling

DECISÃO 03

Backend Modular vs 5 Microserviços Separados

Contexto: A N1 planejou 5 microserviços independentes com API Gateway. Na implementação, o trade-off entre independência de deploy e complexidade operacional ficou claro.

Decisão tomada: **Backend modular monolítico (:8080) + 2 serviços especializados com Kafka (Logistics :8082, Notification :8083).**

Alternativas descartadas:

Alternativa	Por que foi descartada
-------------	------------------------

5 microsserviços com API Gateway	API Gateway adiciona latência e complexidade sem benefício real em dev local. Comunicação REST entre serviços cria acoplamento síncrono — contradiz o requisito de mensageria.
Monolito puro (1 serviço)	Não atende ao requisito de mensageria assíncrona. O fluxo Pedido -> Logística -> Notificação deve ser desacoplado.

Vantagens	Desvantagens / Mitigações
+ Backend centraliza toda lógica de negócio — simples de desenvolver e testar	- Backend é ponto único de falha — aceitável para escopo acadêmico
+ Logística e Notificação são especializados — responsabilidade única, consomem Kafka	- Menos granularidade de escalonamento — irrelevante em dev local
+ Sem API Gateway — menos complexidade operacional, conexões diretas	
+ Bancos separados: backend-db (:5432) e logistics-db (:5435) — independência de dados	
+ Atende ao desacoplamento: Backend publica evento e não espera Logística	
Veredicto: Arquitetura pragmática que entrega o valor do desacoplamento via Kafka sem a complexidade operacional desnecessária de 5 microsserviços em escopo acadêmico.	

DECISAO
04

Virtual Threads (Java 21 — Project Loom)

Contexto: O Logistics Service usa Thread.sleep() para simular delays logísticos (8s e 15s). Com threads de plataforma tradicionais, cada pedido bloquearia uma thread do pool.

Decisão tomada: Virtual Threads habilitadas via spring.threads.virtual.enabled=true em todos os 3 serviços.

Alternativas descartadas:

Alternativa	Por que foi descartada
Thread pool (@Async + Executor)	Pool fixo de threads. Cada Thread.sleep() bloqueia uma thread real. Sob carga, esgota o pool e rejeita pedidos.
CompletableFuture assíncrono	Requer refatoração completa para estilo reativo. Complexidade elevada sem ganho proporcional.
WebFlux / Project Reactor	Paradigma reativo incompatível com JPA síncrono. Reescrita total necessária.

Vantagens	Desvantagens / Mitigações
+ Milhões de Virtual Threads simultâneas sem esgotar threads de SO	- Não recomendado para código CPU-bound — este projeto é I/O-bound (OK)
+ Thread.sleep() libera a Virtual Thread — não bloqueia thread de plataforma	- Bibliotecas não compatíveis podem causar pinning — monitorado via JFR
+ Código imperativo legível (sem callbacks ou Mono/Flux)	
+ Habilitado com 1 linha de configuração	
Veredito: Virtual Threads é a feature certa aqui: código imperativo + delays I/O = benefício direto sem complexidade adicional.	

```
# Logistics Processor – código imperativo, sem callback:
Thread.ofVirtual().start(() -> {
    Thread.sleep(shippingDelayMs); // libera VT, não bloqueia SO
    producer.publishStatus(orderId, userId, "SHIPPED");
    Thread.sleep(deliveryDelayMs);
    producer.publishStatus(orderId, userId, "DELIVERED");
});
```

DECISAO
05

WebSocket STOMP vs Polling vs SSE

Contexto: O frontend precisa exibir mudanças de status do pedido em tempo real sem recarregar a página.

Decisão tomada: Spring WebSocket + STOMP no Notification Service; SockJS + @stomp/stompjs no React.

Alternativas descartadas:

Alternativa	Por que foi descartada
Polling (setTimeout + GET /orders)	Gera N requisições por segundo por usuário. Ineficiente, com atraso proporcional ao intervalo.

Server-Sent Events (SSE)	Unidirecional apenas. Não suporta SockJS fallback. Sem ecossistema Spring STOMP.
Long Polling	Complexo de implementar. Maior latência. Obsoleto diante do WebSocket nativo nos browsers modernos.

Vantagens	Desvantagens / Mitigações
+ Push bidirecional — servidor empurra sem o cliente solicitar	- 1 conexão persistente por usuário ativo — aceitável para escala do projeto
+ STOMP permite subscrição por destino (/topic/notifications/{userId})	- Requer tratamento de reconexão no frontend — implementado no hook useNotifications.js
+ SockJS garante fallback para ambientes sem WebSocket nativo	
+ Menor tráfego: 1 conexão persistente vs N requisições HTTP	
+ Latência < 1s desde a publicação Kafka até a exibição no browser	
Veredito: WebSocket STOMP é o padrão correto para notificações em tempo real. Notification Service completamente stateless — sem banco de dados.	

```
// useNotifications.js — conexão e subscrição
const client = new Client({
  websocketFactory: () => new SockJS('http://localhost:8083/ws'),
  onConnect: () => {
    client.subscribe(`/topic/notifications/${userId}`,
      (msg) => {
        const notif = JSON.parse(msg.body);
        dispatch({ type: 'ADD', payload: notif });
        window.dispatchEvent(
          new CustomEvent('teestore-order-update'));
      }
    );
  }
});
```

DECISAO

06

JWT Stateless + Redis (Refresh Token + Blacklist)

Contexto: O backend precisa de autenticação stateless para suportar múltiplas instancias, mas também precisa de logout efetivo — invalidar tokens antes da expiração.

Decisão tomada: **JWT access token (1h, stateless) + Refresh token no Redis (7 dias) + Blacklist no Redis.**

Alternativas descartadas:

Alternativa	Por que foi descartada
Sessão HTTP (HttpSession)	Stateful — cada instancia do backend precisa compartilhar sessão. Incompatível com escalonamento horizontal.
JWT puro sem Redis	Logout impossível — token continua valido ate expirar. Token interceptado da acesso por ate 1h.
OAuth2 / Keycloak	Overhead de infraestrutura desnecessario para o escopo academico.

Vantagens	Desvantagens / Mitigacoes
+ Access token stateless: zero query no banco por requisicao	- Redis e um ponto de dependencia para autenticao — mitigado com healthcheck no Docker Compose
+ Refresh token rotacionado a cada uso — rotacao automatica de sessão	- Blacklist cresce com tokens revogados — TTL garante limpeza automatica
+ Blacklist no Redis invalida tokens antes da expiração (logout efetivo)	
+ Redis TTL automatico — sem cleanup manual de tokens expirados	
+ Interceptor Axios renova o token automaticamente no frontend	
Veredicto: JWT + Redis e o padrao da industria para APIs REST modernas. Atende stateless, logout efetivo e rotacao de sessão sem overhead de OAuth2.	

```
# Redis keys utilizadas:
refresh:{token} -> userId (TTL: 7 dias)
blacklist:{token} -> '1' (TTL: 1h = tempo do access token)
cart:{userId} -> JSON (TTL: 30 dias)

# Interceptor Axios - renovacao automatica do token
if (err.response?.status === 401) {
  const { data } = await refreshToken();
  setToken(data.token);
  return instance(err.config); // retry
}
```

DECISAO

07

React 18 + Vite 5 — Frontend

Contexto: O frontend precisa ser uma SPA responsiva com suporte a WebSocket, roteamento client-side e painel administrativo completo.

Decisão tomada: **React 18 com Vite 5, CSS Modules, Axios, @stomp/stompjs + SockJS.**

Alternativas descartadas:

Alternativa	Por que foi descartada
Angular	Curva de aprendizado alta (RxJS, módulos, decorators). Overkill para o escopo.
Vue.js 3	Opção válida, mas equipe tem mais familiaridade com React. Ecossistema STOMP menor.
Next.js (SSR)	Server-Side Rendering desnecessário para e-commerce que requer autenticação em todas as rotas.

Vantagens	Desvantagens / Mitigações
+ Vite: HMR < 50ms — ciclo de desenvolvimento rápido	- SPA sem SSR: SEO limitado — irrelevante para e-commerce autenticado
+ React 18 Concurrent Mode: UI fluida durante updates de estado	- Bundle size maior que alternativas mais leves — mitigado pelo lazy loading de rotas
+ CSS Modules: escopo de estilos por componente — sem conflitos globais	
+ Hooks personalizados (useNotifications, useCart, useAuth) — lógica reutilizável	
+ useCallback + useEffect em Orders.jsx evitam re-renders desnecessários	
Veredicto: React + Vite é a combinação mais produtiva para o time. CSS Modules e hooks resultaram em código modular e de fácil manutenção.	

Síntese — Por que estes Design Patterns?

Os padrões abaixo foram escolhidos porque cada um resolve um problema concreto. Não foram aplicados para 'cumprir a rubrica' — foram escolhidos porque a solução sem eles seria tecnicamente pior.

Pattern	Problema que resolve	Onde	Consequência sem ele
DI (Construtor)	Classes difíceis de testar isoladamente	Todos os @Service	Testes precisariam de @SpringBootTest — lentos e dependentes de infra
Strategy (switch expression)	Lógica condicional espalhada em if/else de status	StatusEventConsumer	Cada novo status altera múltiplos arquivos — violação do OCP
Observer (Kafka + STOMP)	Backend precisa notificar Logistics e Frontend sem conhecê-los	Toda camada de mensageria	Acoplamento direto: Backend chama Logistics via REST — ponto único de falha
Template Method	Ciclo logístico com esqueleto fixo mas delays configuráveis	LogisticsProcessor	Delays hardcoded — cada mudança requer recompilação e redeploy

Repository (JPA)	Logica SQL misturada com logica de negocio	Todos os @Service	Services com queries SQL — impossível trocar banco ou mockar em testes
Idempotencia	Kafka entrega at-least-once — reprocessamento gera duplicatas	LogisticsProcessor	Cada reentrega Kafka cria pedido logístico duplicado — dados corrompidos

Princípio central adotado: O pacote domain/ não importa nada do Spring Framework. Entidades e regras de negócio são puro Java. Isso é o que torna os 58 testes unitários do Backend independentes de infraestrutura — rodam em milissegundos, sem Docker, sem banco. Essa é a essência da Clean Architecture aplicada ao projeto.

Projeto Integrador — Desenvolvido por Victor Hugo, Josue Felix e Guilherme Bastos
Curso Analise e Desenvolvimento de Sistemas — 4 Período · PUC Goiás · 2026/1